Machine learning

CLASSIFICATION – K NEAREST NEIGHBORS (K-NN)

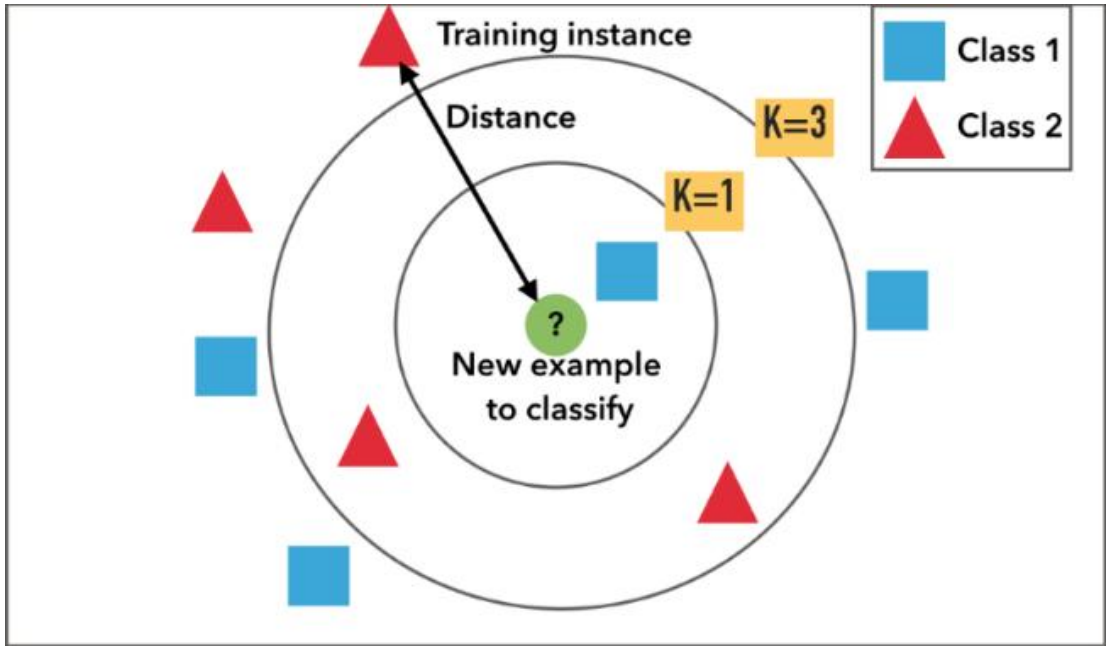
k-Nearest Neighbor Classification (kNN)

Unlike all the previous learning methods, kNN does not build model based on the training data

1. To classify a test instance **d**, define k-neighborhood **P** as **k nearest neighbors of d**
2. Count number n of training instances in P that belong to class c_j
3. Estimate $Pr(c_j|d)$ as n/k

No training is needed.

Classification time is linear in training set size

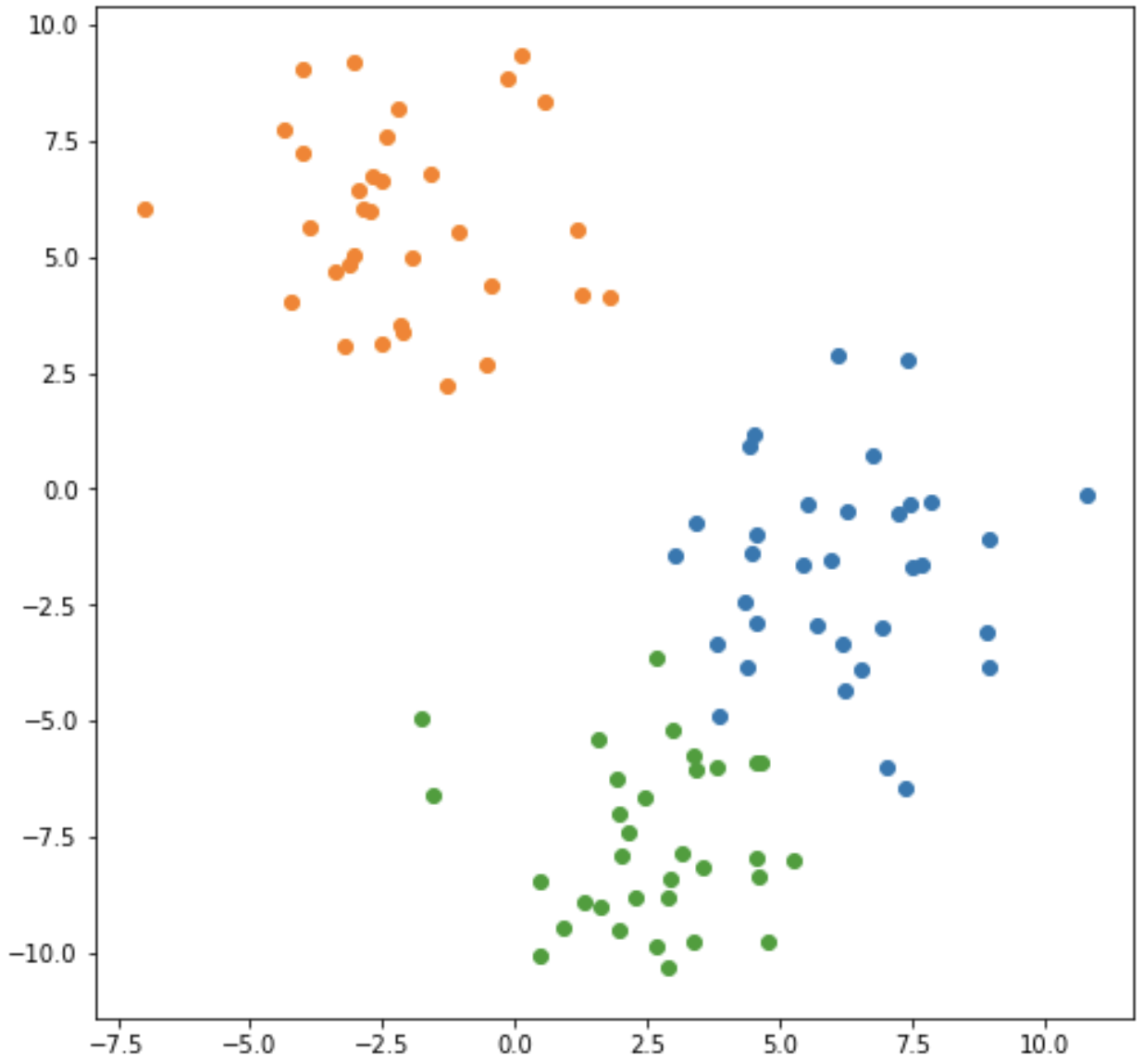


kNN Algorithm

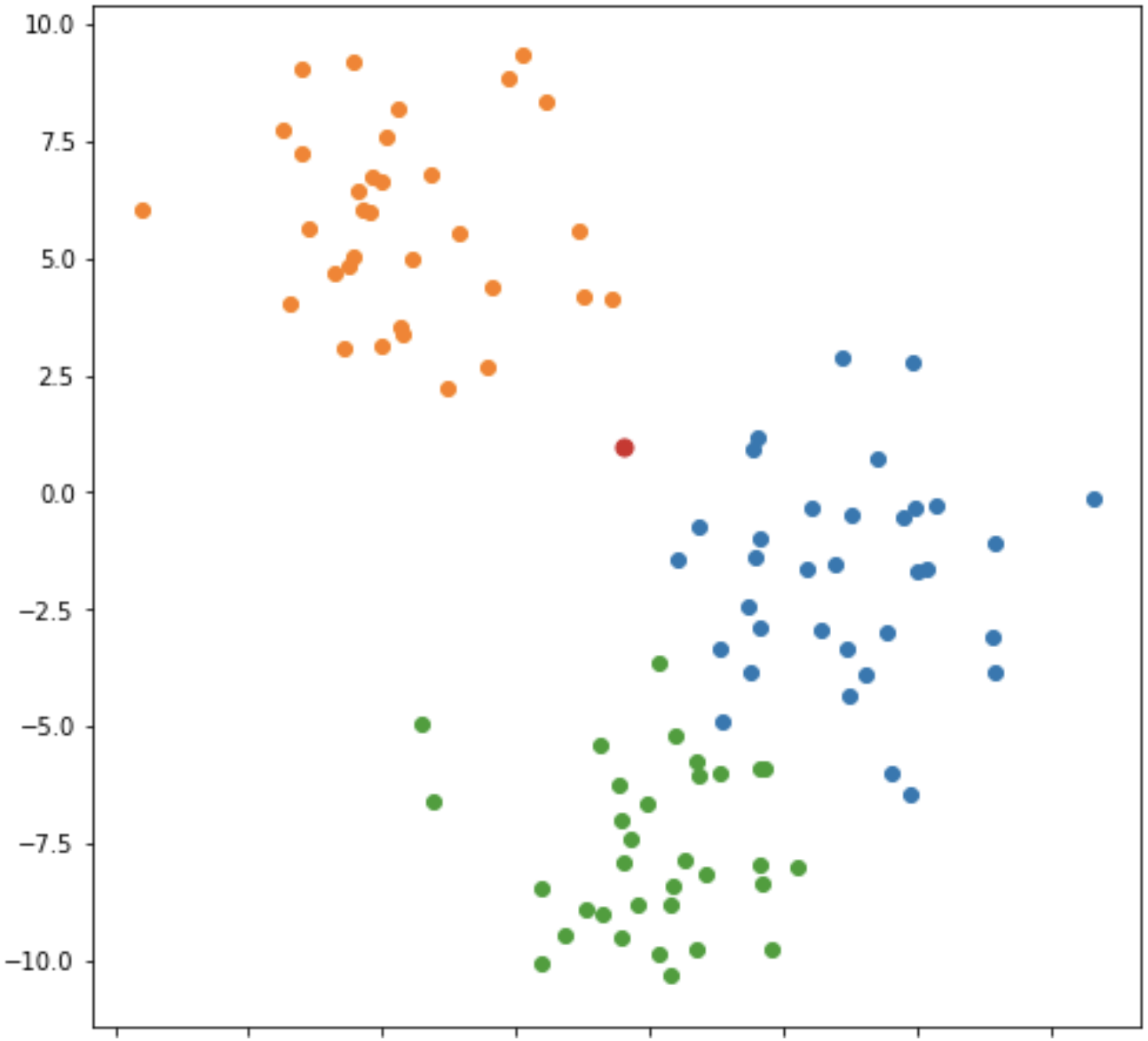
Algorithm kNN(D, d, k)

1 Compute the distance between d and every example in D;
2 Choose the k examples in D that are nearest to d, denote the set by P ($\subseteq D$);
3 Assign d the class that is the most frequent class in P (or the majority class);

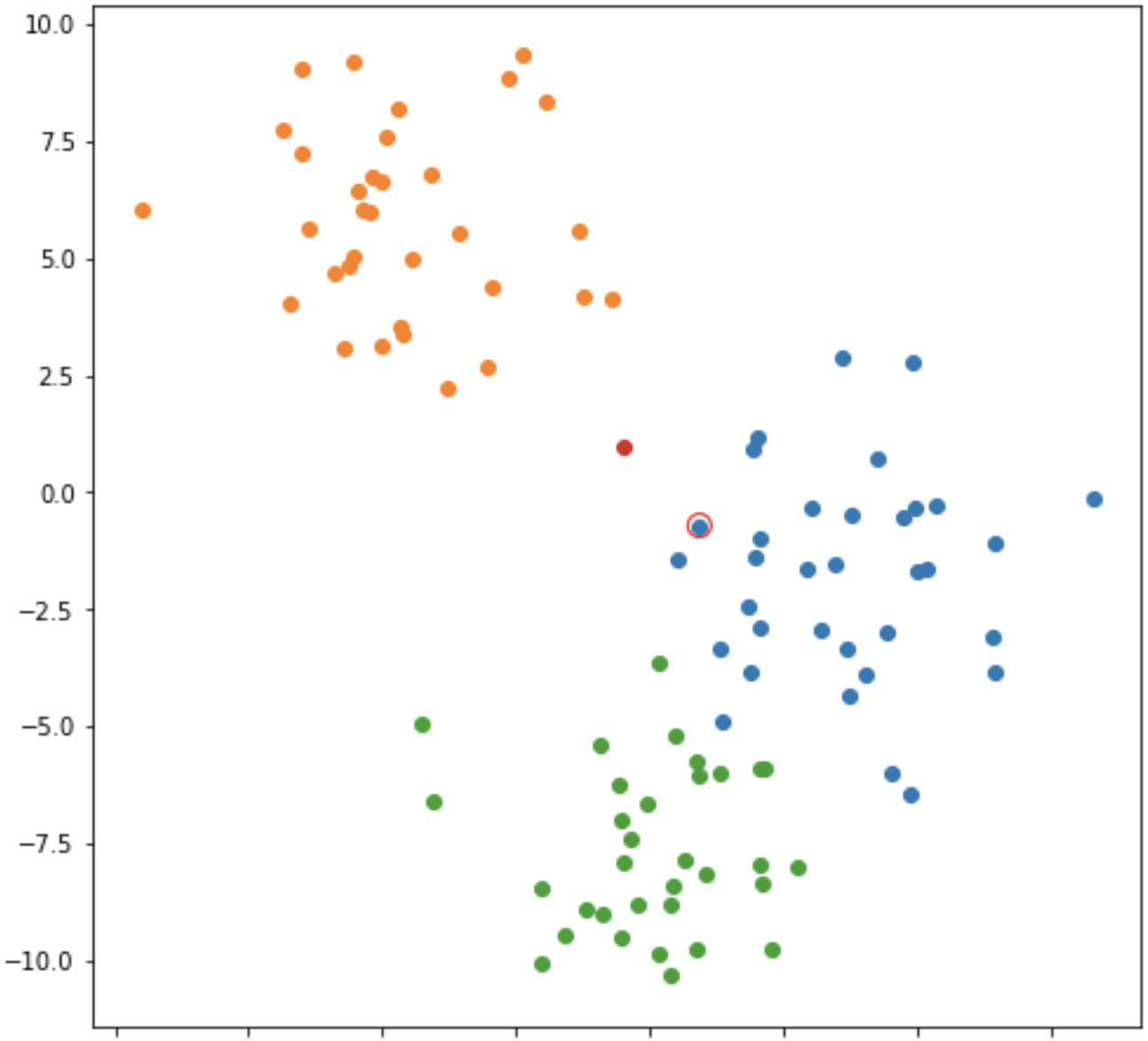
- k is usually chosen empirically via a validation set or cross-validation by trying a range of k values.
- Distance function is crucial but depends on applications.



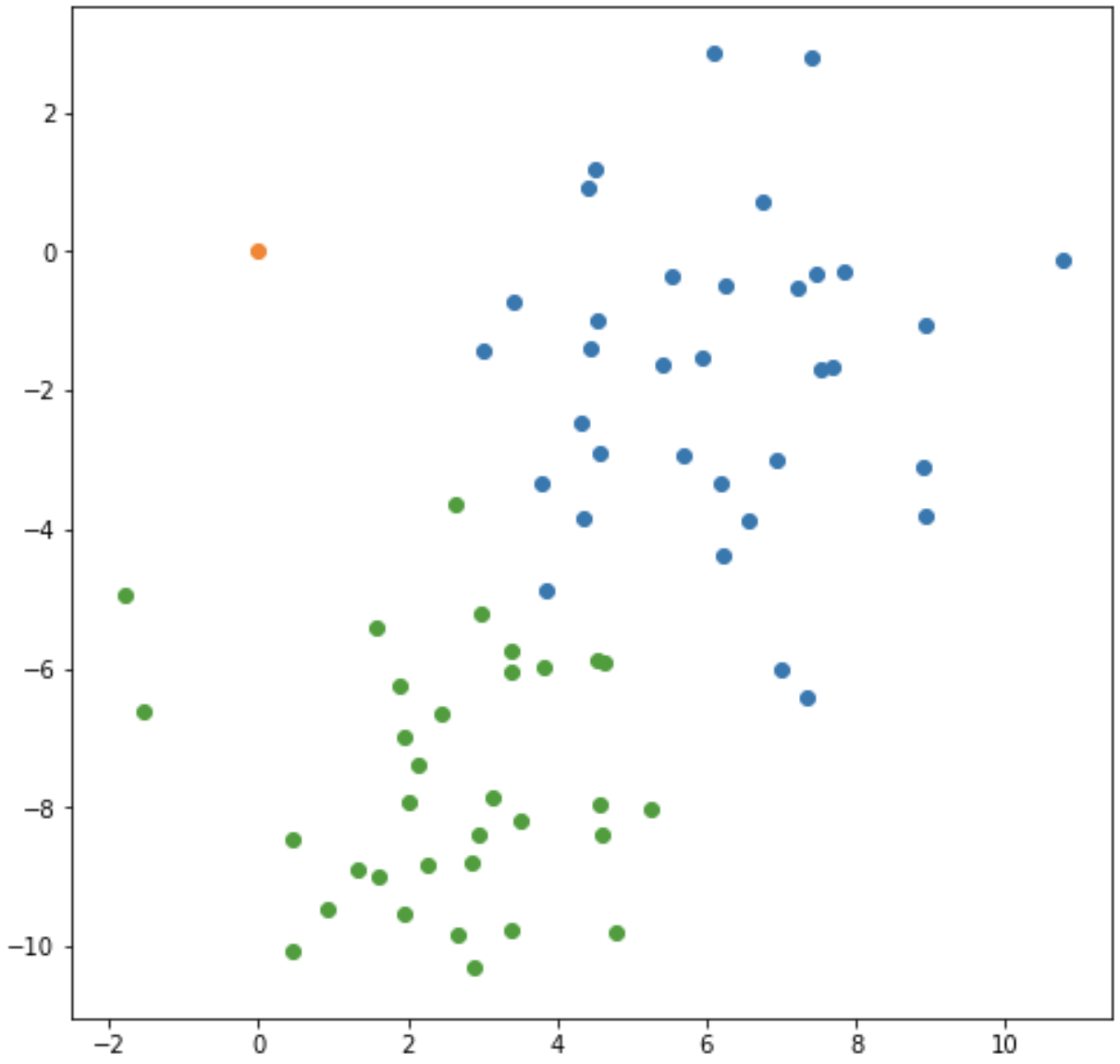
K-NN Example



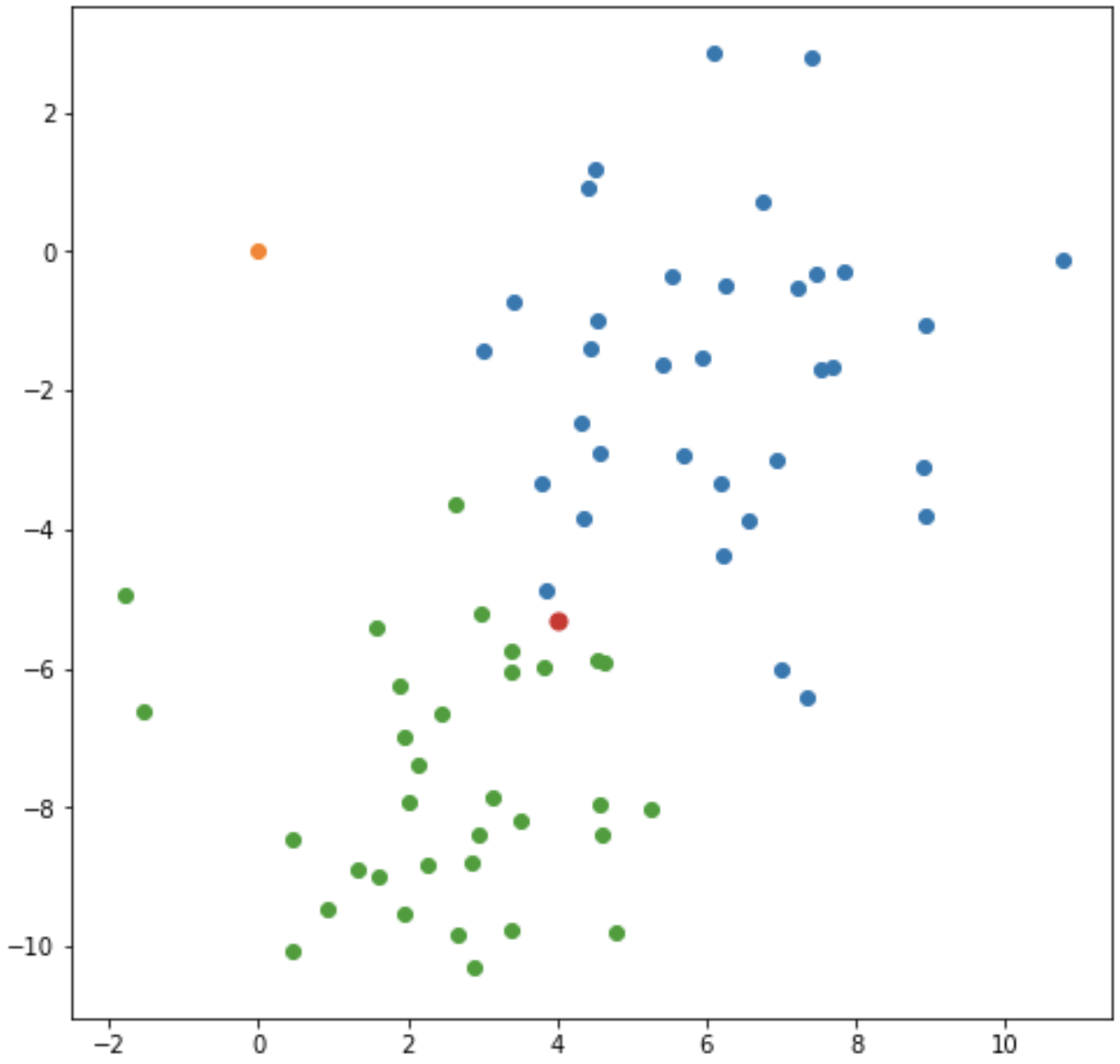
How to classify the red dot?



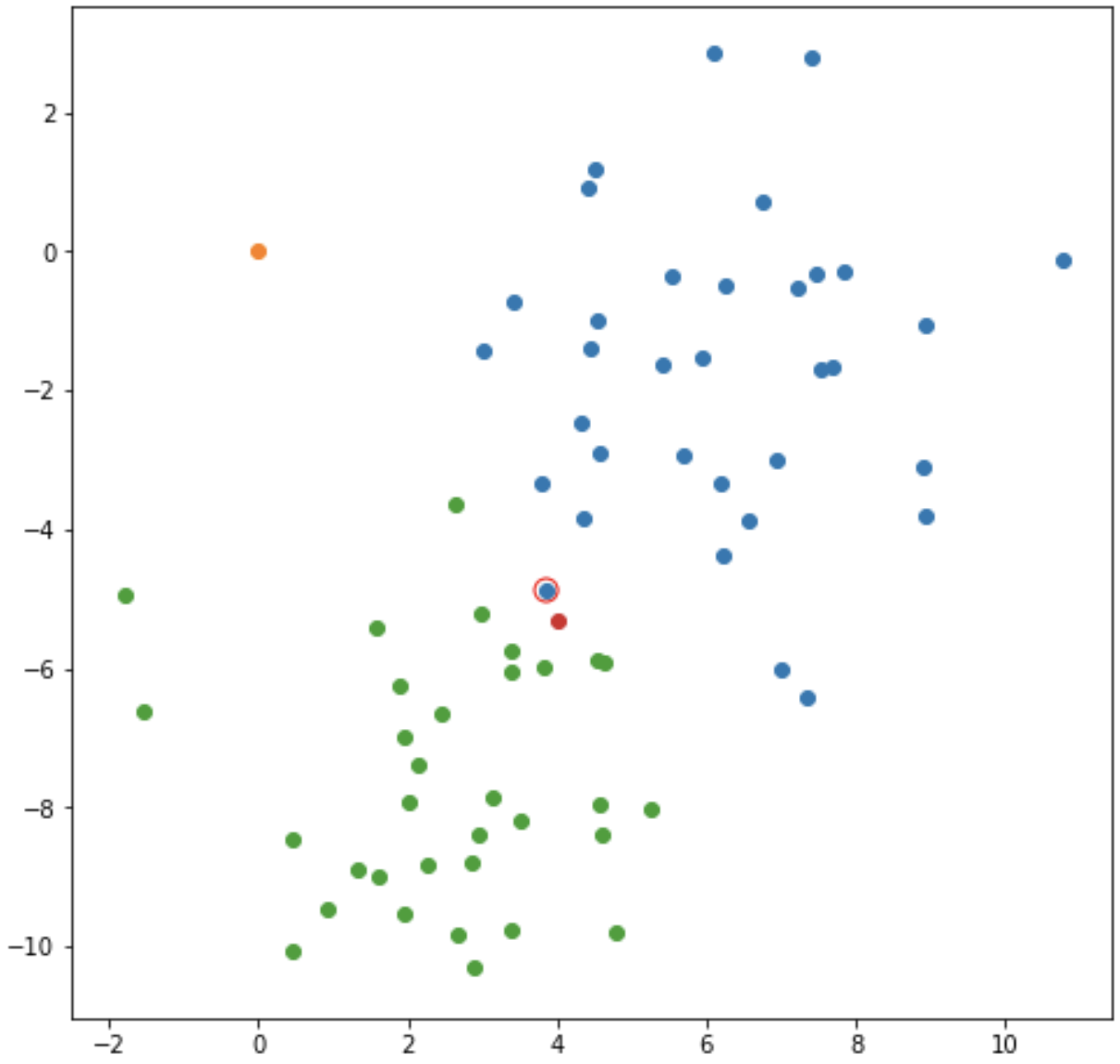
Based on the nearest neighbor



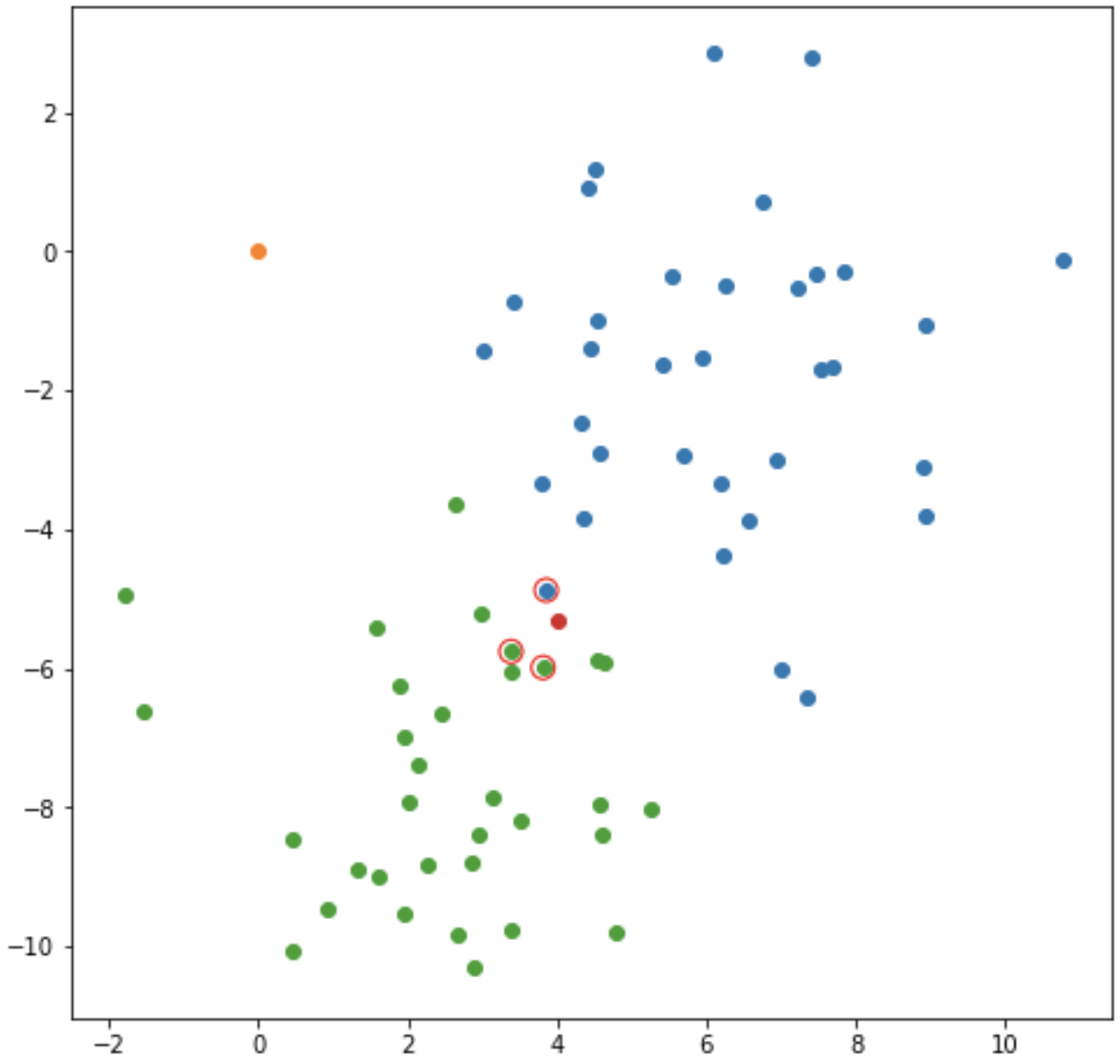
Zoom in (boundary of green and blue)



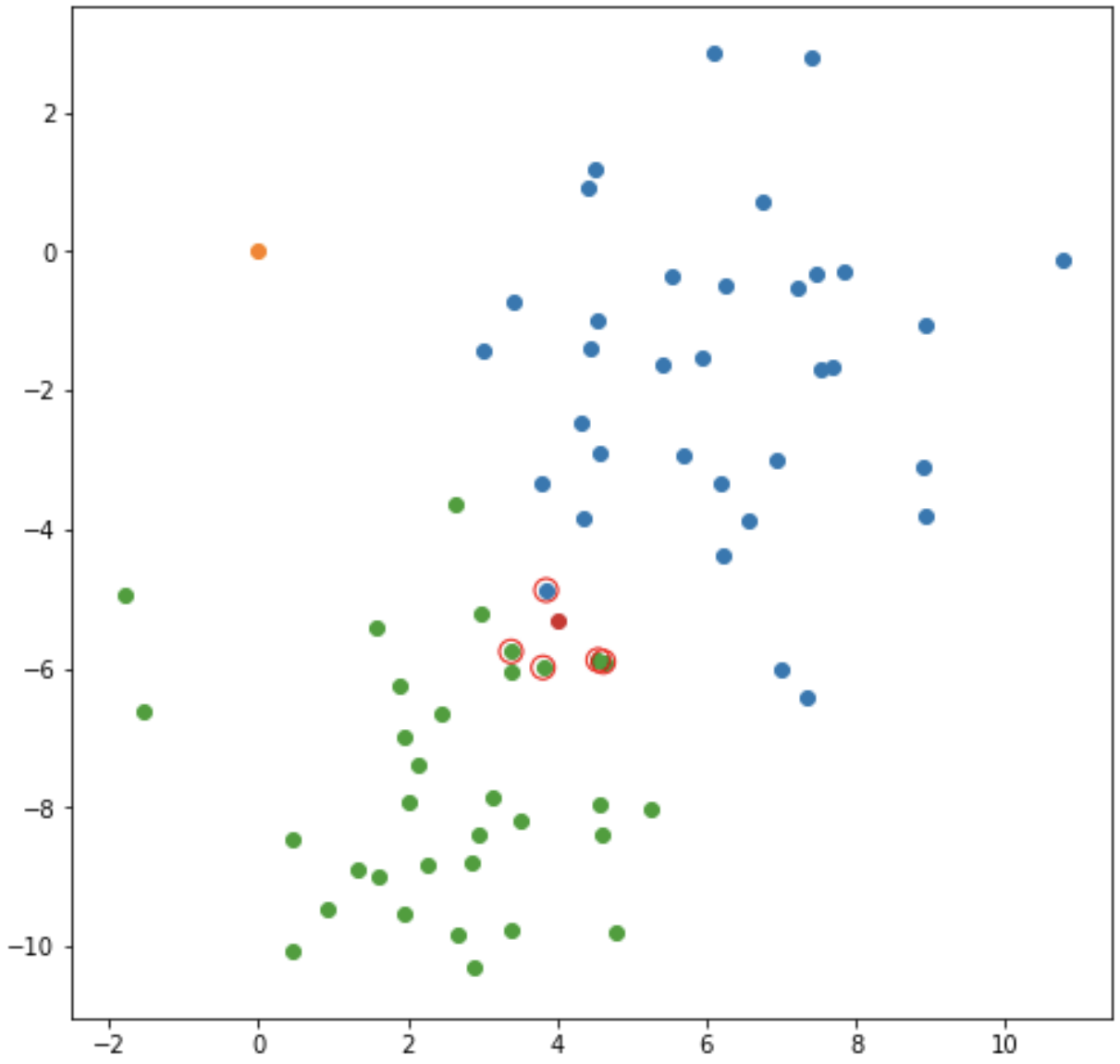
New point of interest



Closest point is a blue one



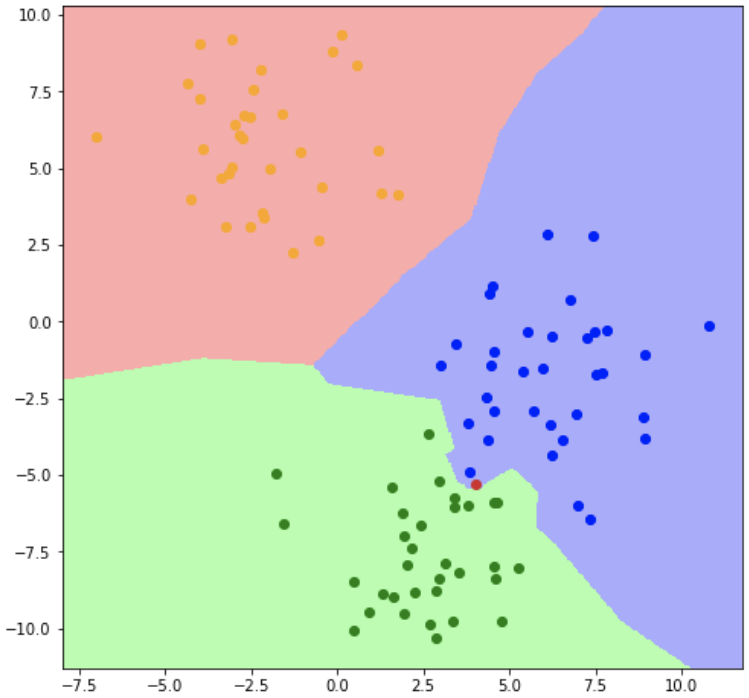
Why not 3 nearest neighbors?



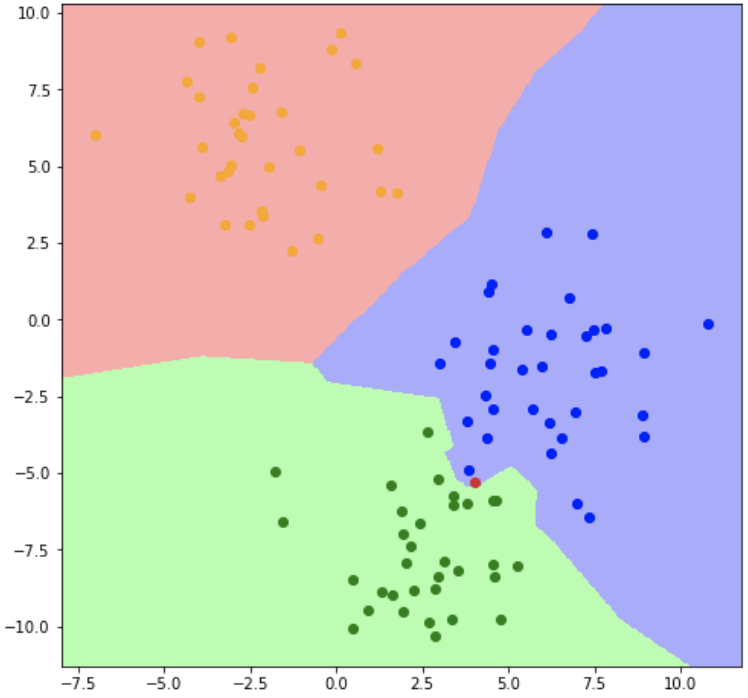
Or 5?

Boundaries

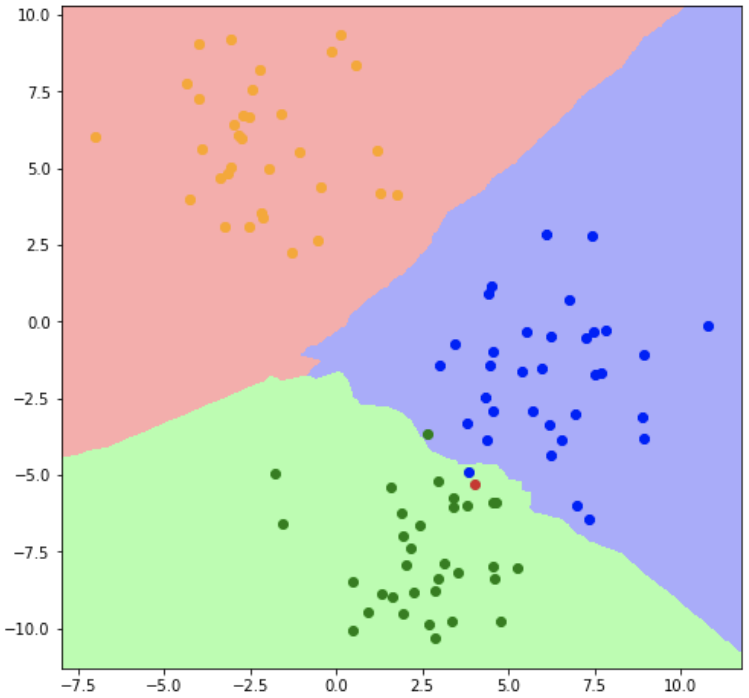
K=1



K=3

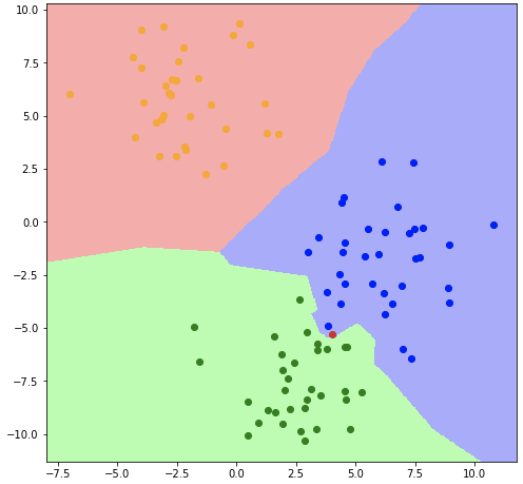


K=5



Pros:

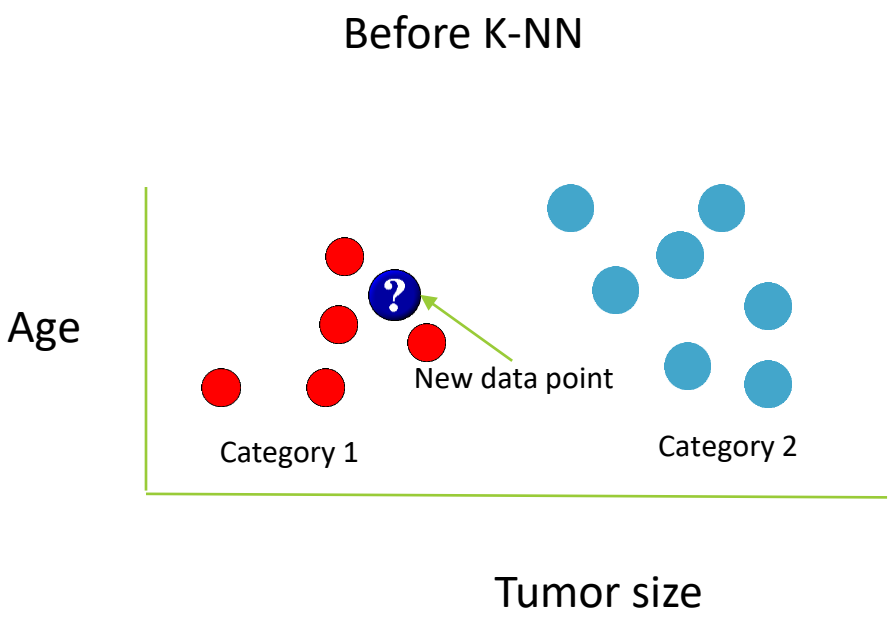
- kNN can deal with complex and arbitrary decision boundaries
- Despite its simplicity, researchers have shown that the classification accuracy of kNN can be quite strong and, in many cases, as accurate as elaborated methods

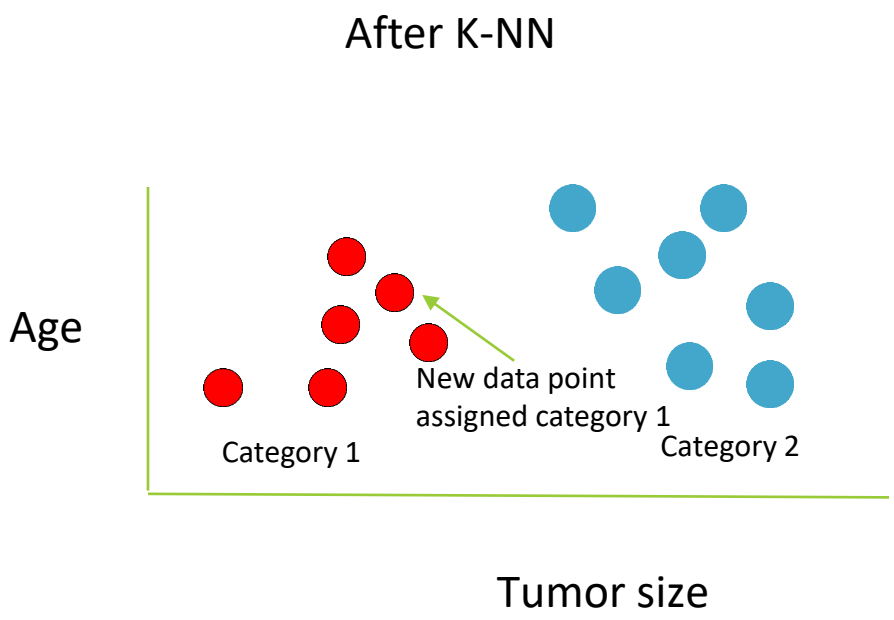


Cons:

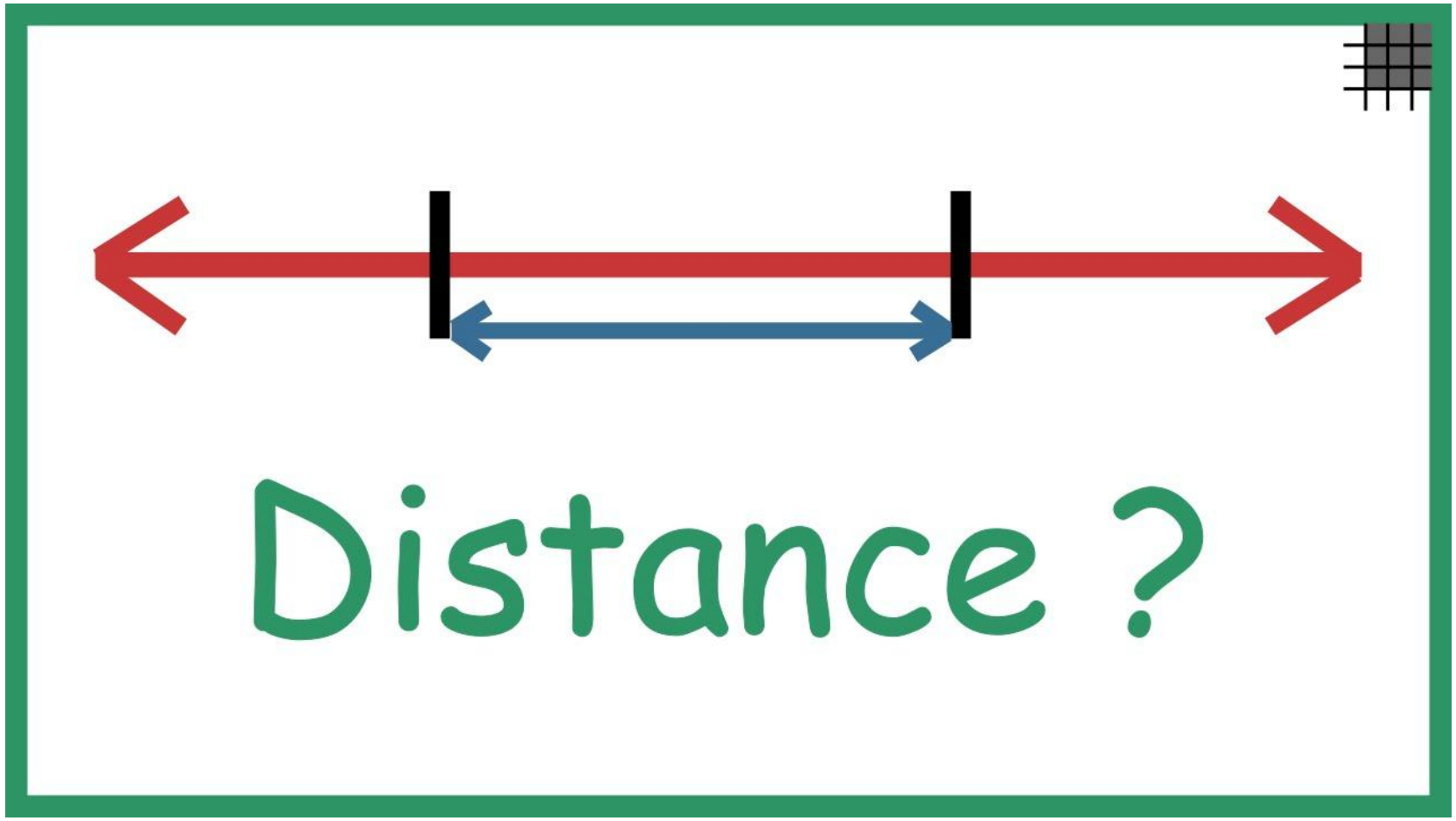
- kNN is slow at classification time
- kNN does not produce an understandable model

K nearest neighbors

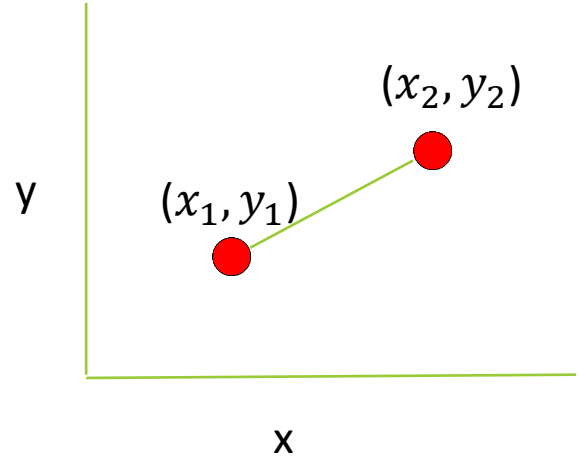




What is considered close?

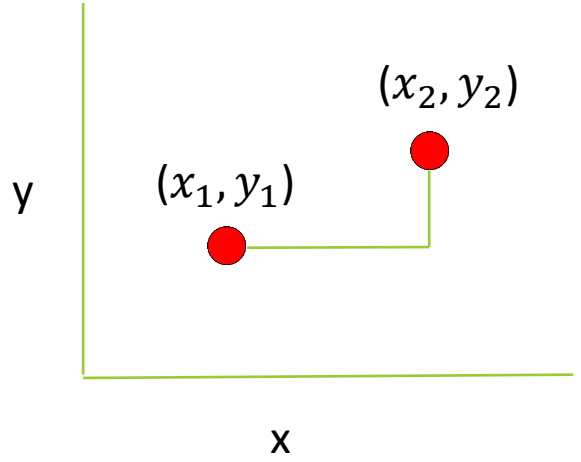


Euclidian distance (L2)



Dist= $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Manhattan distance (L1)



Dist=$|x_1 - x_2| + |y_1 - y_2|$

"The distance between two points in a grid based on a strictly horizontal and/or vertical path (that is, along the grid lines), as opposed to the diagonal or "as the crow flies" distance. The Manhattan distance is the simple sum of the horizontal and vertical components, whereas the diagonal distance might be computed by applying the Pythagorean theorem." (Wiki)

Hamming distance

In the instance of categorical variables, the Hamming distance should be used.

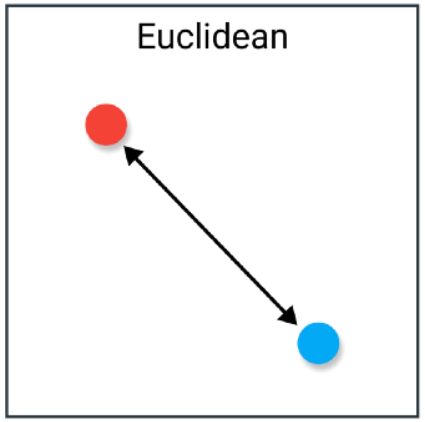
This metric measures the number of changes between two points:

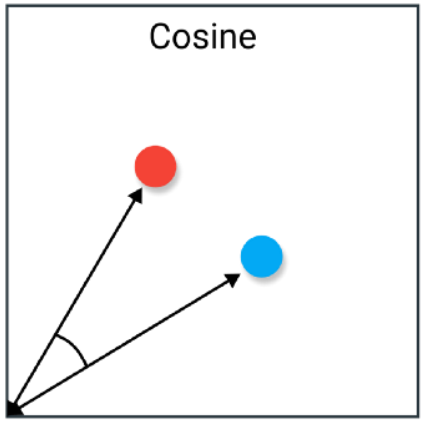
P_1=$(x_1, x_2, .., x_k)$ P_2=$(y_1, y_2, .., y_k)$

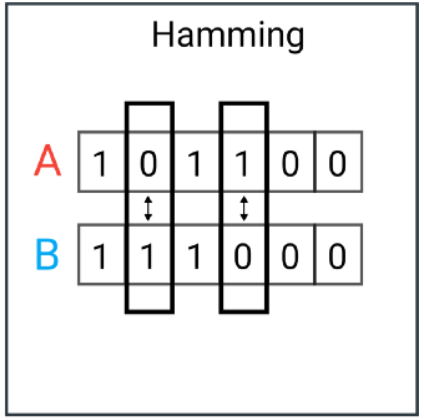
D(x_i, y_i) = 0 if $x_i == y_i$, otherwise 1

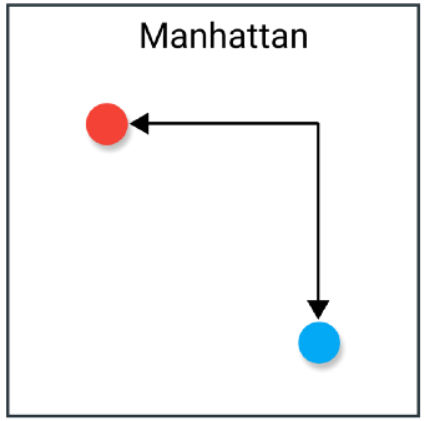
Hamming distance(P_1, P_2) = $\sum_{i=1}^{k} D(p_1[i], p_2[i])$

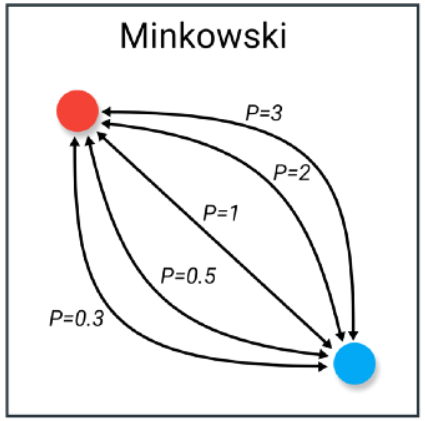
Distances

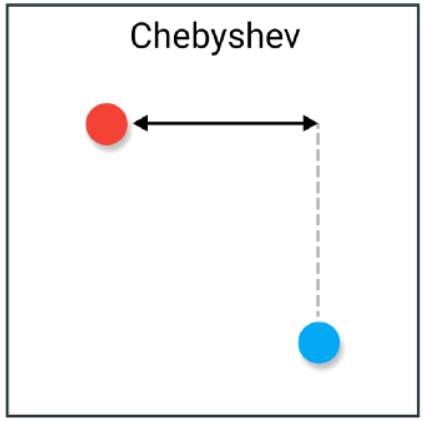

Euclidean

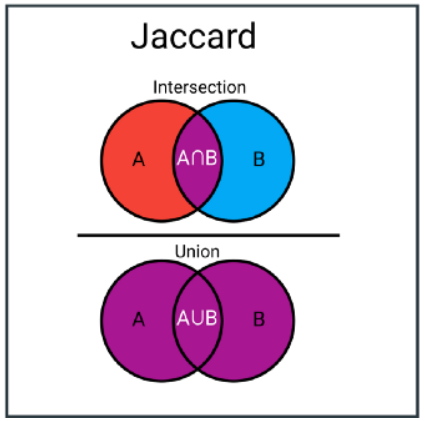

Cosine

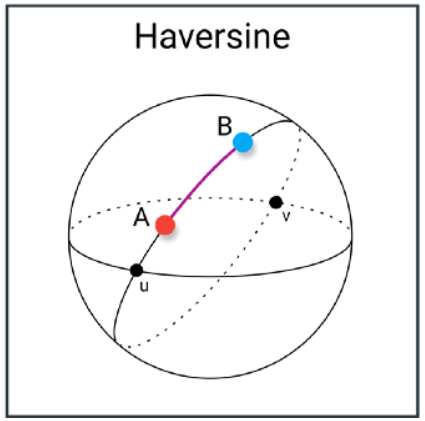

Hamming
A
1 0 1 1 0 0
B
1 1 1 0 0 0

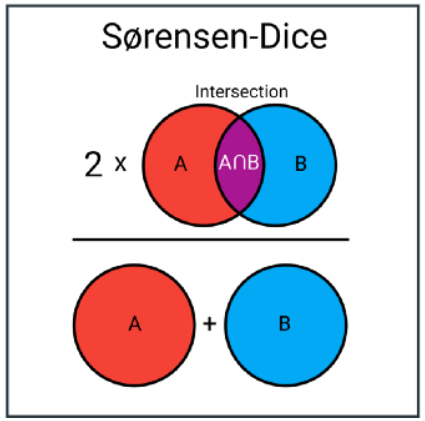

Manhattan


Minkowski
P=3
P=2
P=1
P=0.5
P=0.3


Chebyshev

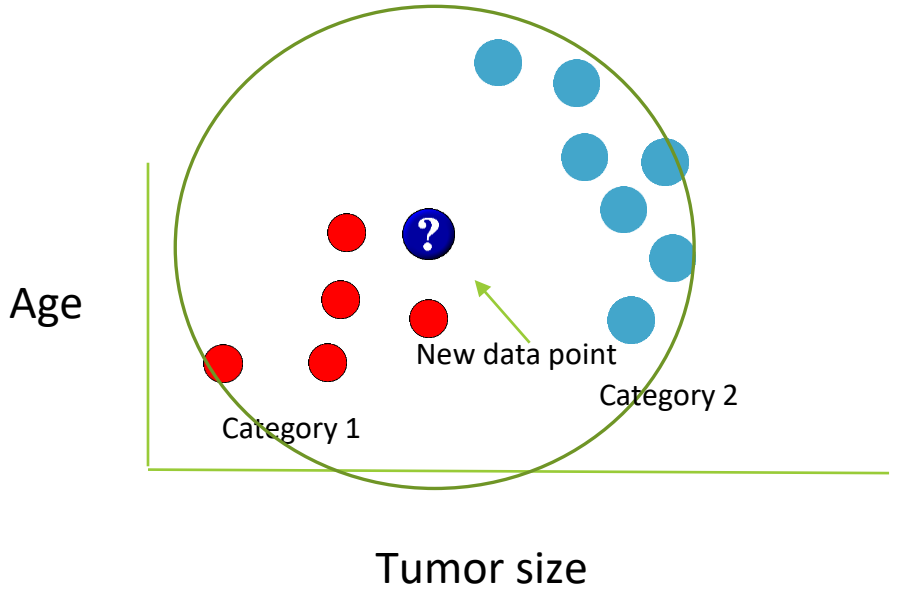

Jaccard
Intersection
A
A∩B
B
Union
A
AUB
B


Haversine
B
A
v
u


Sørensen-Dice
Intersection
2 x
A
A∩B
B
A
+
B

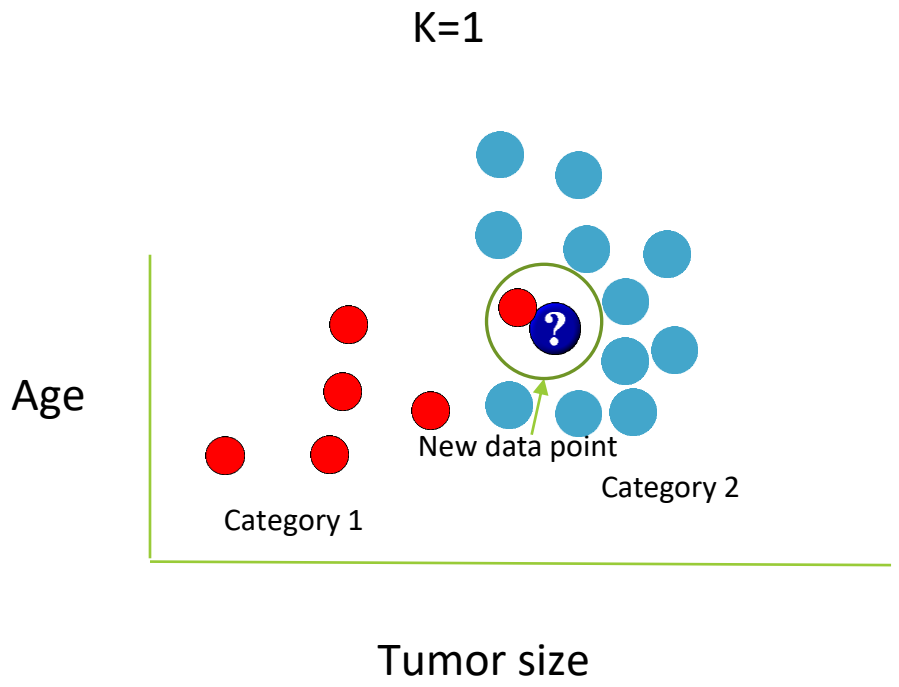
Finding the optimal number of neighbors

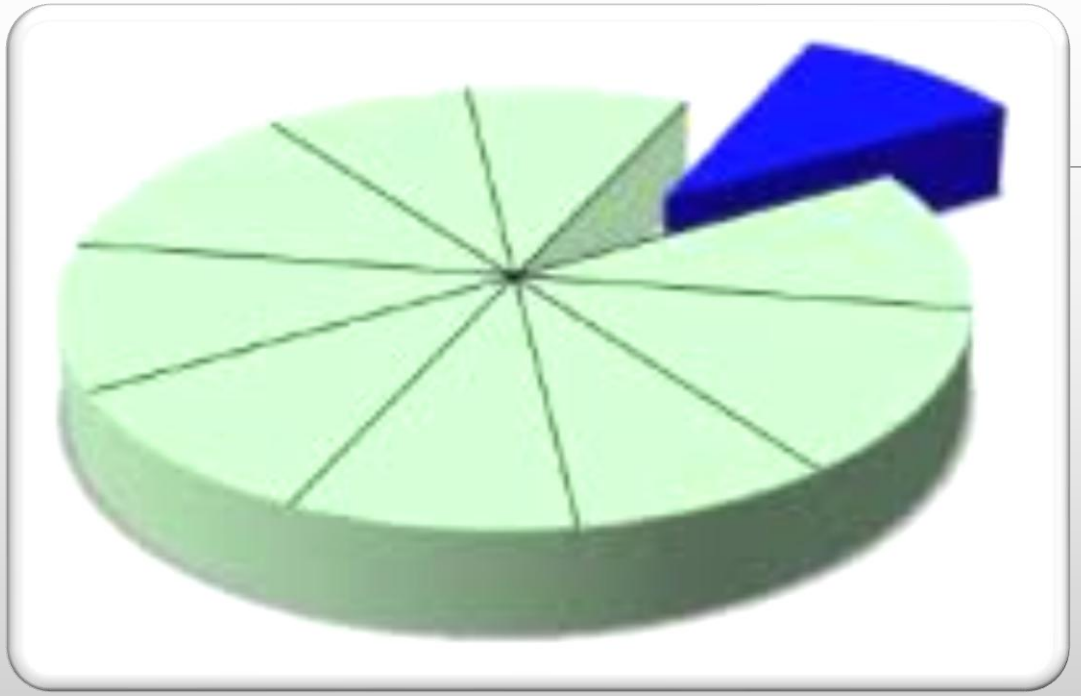
Too big – neighborhood might include points from other neighborhoods



Finding the optimal number of neighbors

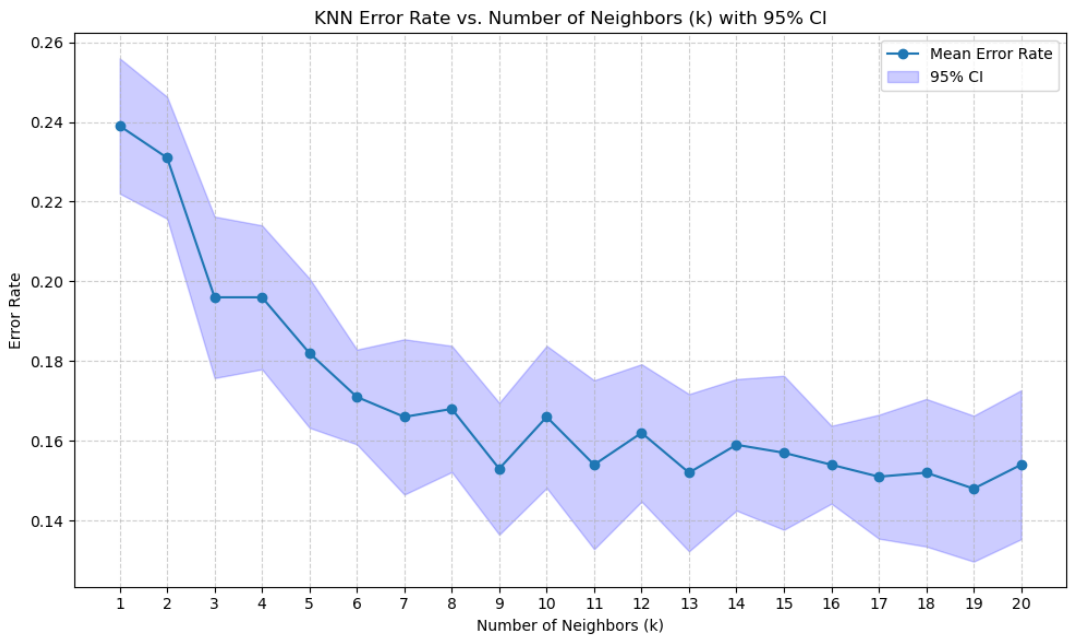
Too small – sensitive to noise points





Finding the optimal number of neighbors

JUST RIGHT – **EXPERIMENT** WITH DIFFERENT K VALUES

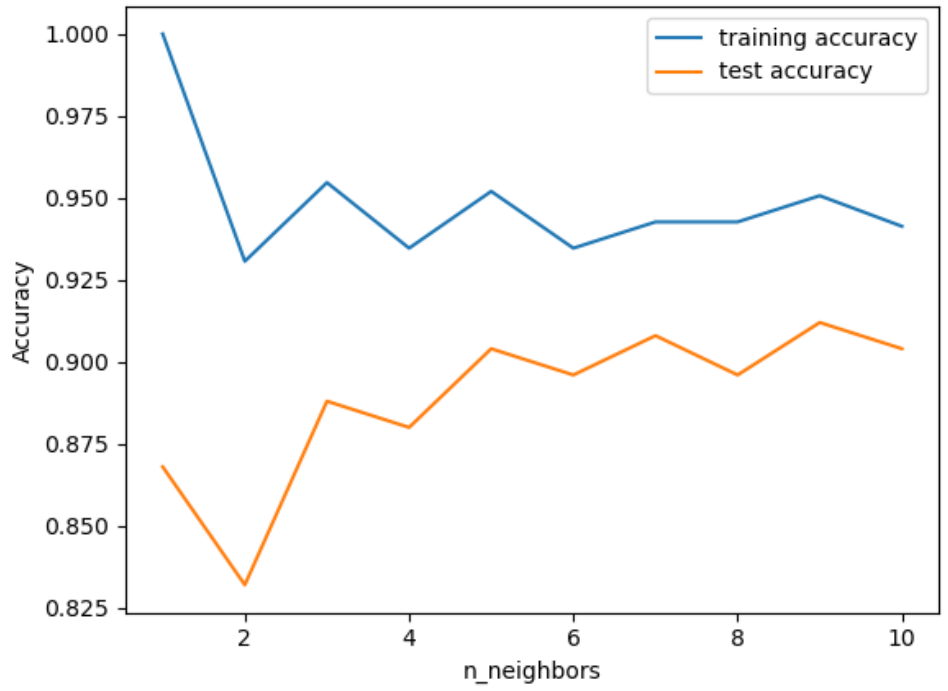



```python
k_range = range(1, 21)
mean_errors = []
std_errors = []

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for k in k_range:
    knn_cv = KNeighborsClassifier(n_neighbors=k)
    # Use cross_val_score with scoring='accuracy'
    scores = cross_val_score(knn_cv, X, y, cv=cv, scoring='accuracy')
    error = 1 - scores
    mean_errors.append(error.mean())
    std_errors.append(error.std())

mean_errors = np.array(mean_errors)
std_errors = np.array(std_errors)
ci = 1.96 * std_errors / np.sqrt(cv.get_n_splits())
```

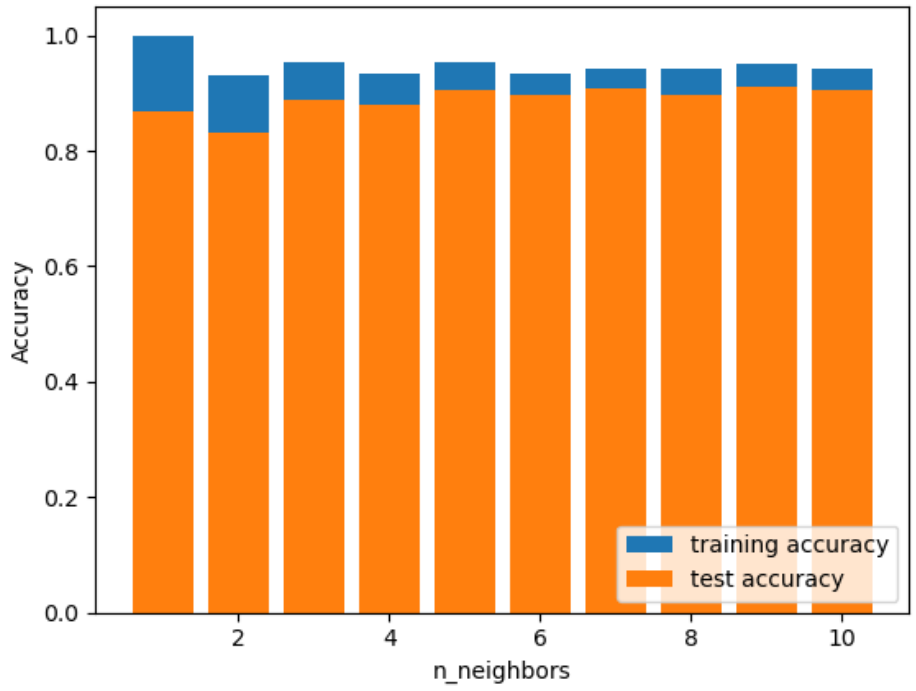
Elbow method

Importance of correct vizualization

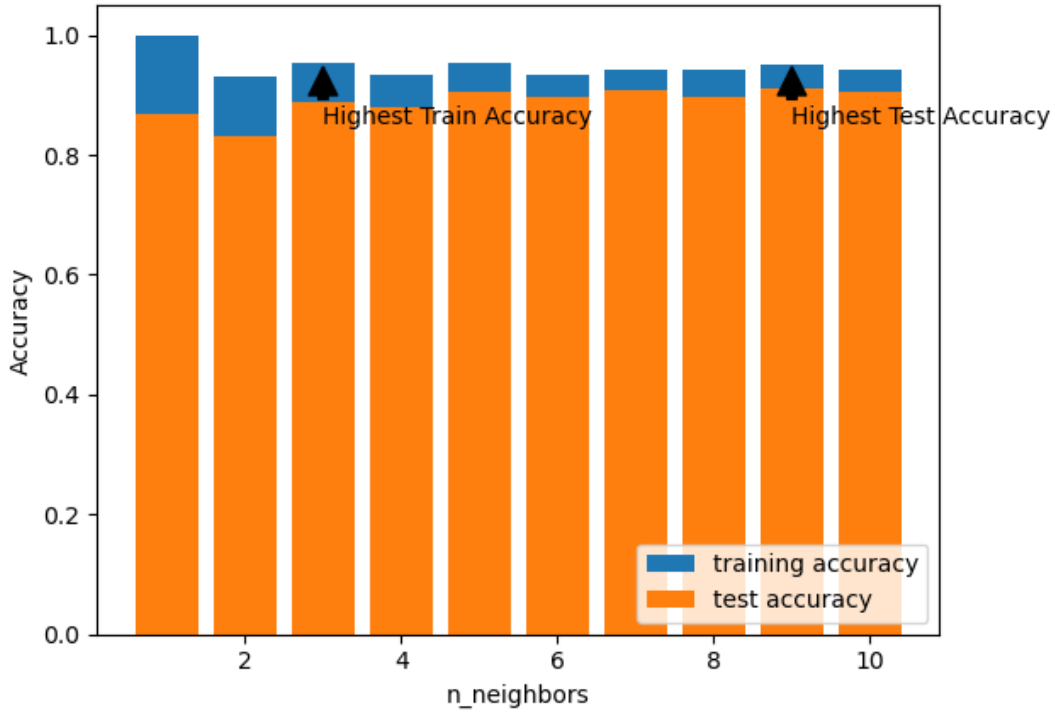
PLOT



BAR



```
plt.bar(neighbors_settings, training_accuracy, label="training accuracy")
plt.bar(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend(loc='lower right')
# show values for which train accuracy is highest and test accuracy is highest
plt.annotate('Highest Train Accuracy', xy=(3, 0.95), xytext=(3, 0.85),
arrowprops=dict(facecolor='black', shrink=0.05),)
plt.annotate('Highest Test Accuracy', xy=(9, 0.95), xytext=(9, 0.85),
arrowprops=dict(facecolor='black', shrink=0.05),)
plt.show()
```

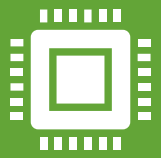


Even better….

Knn - Pros

- It is extremely easy to implement
- It is lazy learning algorithm and therefore requires no training prior to making real time predictions.
- Since the algorithm requires no training before making predictions, new data can be added seamlessly.
- There are only two parameters required to implement KNN:
  - The value of K
  - The distance function (e.g., Euclidean or Manhattan)

Knn - Cons



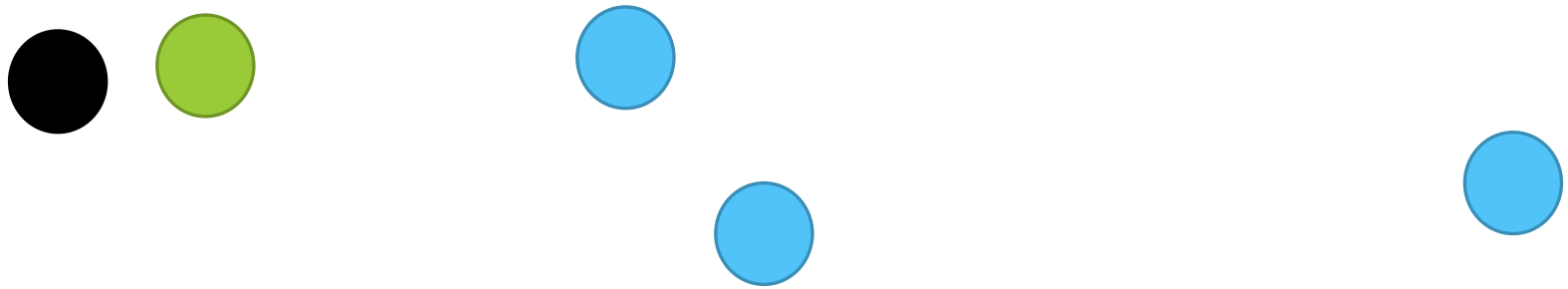
The KNN algorithm **doesn't work well with high dimensional data** because with large number of dimensions, it becomes difficult for the algorithm to calculate distance in each dimension.

The KNN algorithm has a **high prediction cost for large datasets**.
This is because in large datasets the cost of calculating distance between new point and each existing point becomes higher.

Radius Neighbors Classifier

Motivation

- **What happens if the k closest neighbors are not that close?**
- **Set a radius instead**
  - dense regions of the feature space will contribute more information, and sparse regions will contribute less information



Coding example

```
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]
from sklearn.neighbors import RadiusNeighborsClassifier
neigh = RadiusNeighborsClassifier(radius=1.0)
neigh.fit(X, y)
print(neigh.predict([[1.5]]))
#[0]
print(neigh.predict_proba([[1.5]]))
#[[0.5 0.5]]
print(neigh.predict_proba([[1.0]]))
#[[0.66666667 0.33333333]]
```

Approximate nearest neighbor (ANN)

Nearest Neighbor Search — The Big Idea

Goal: Find the point(s) closest to a given query point

Why?

- Image similarity
- Text document matching
- Recommendation systems

🧠 **Exact NN**: Finds the *true* nearest point

⚡ **Approximate NN**: Finds a *close enough* neighbor — much faster!

Why Go Approximate?

High-dimensional data is hard!

- Curse of dimensionality → brute-force is too slow

ANN trades a little accuracy for a lot of speed

- From seconds → milliseconds per query

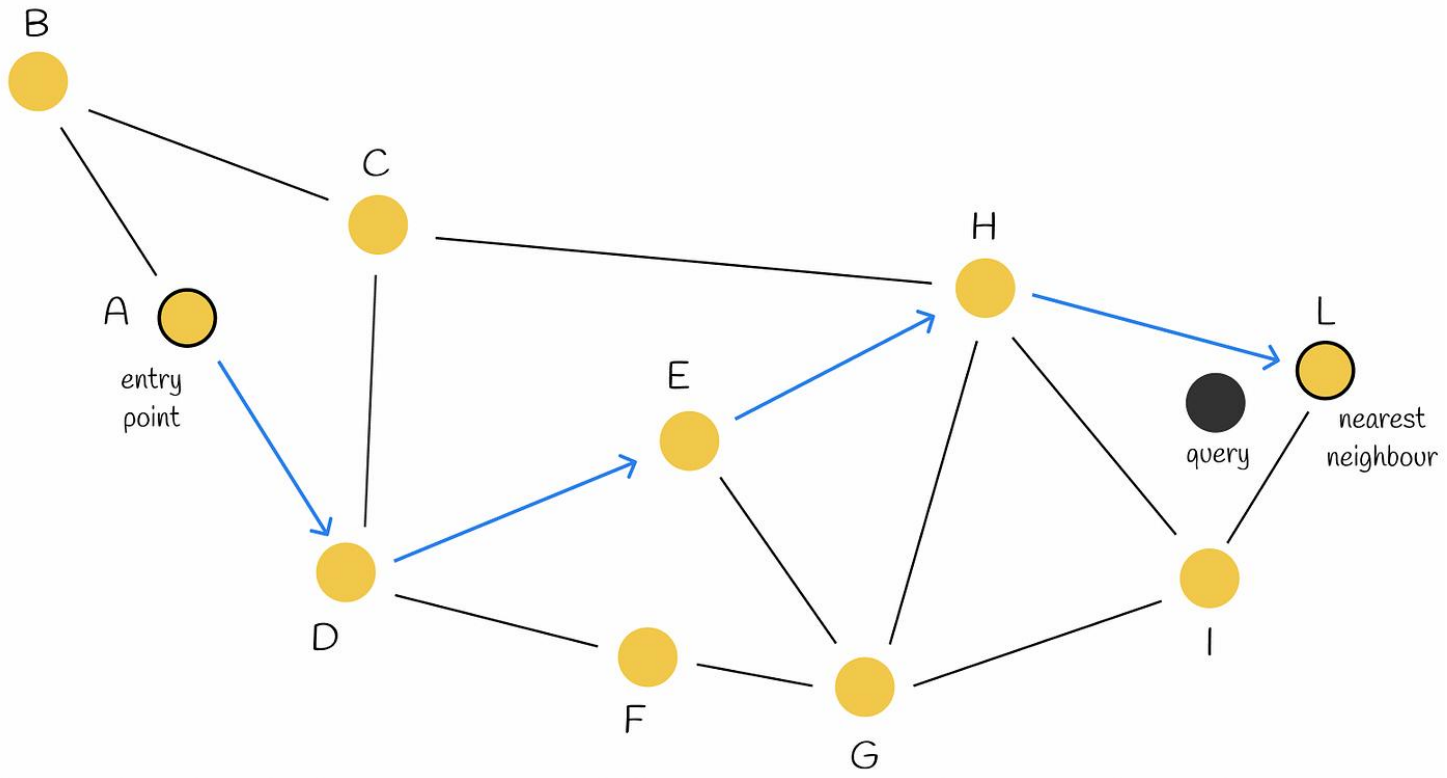
Popular ANN Algorithms

1. **Locality Sensitive Hashing (LSH)**
   - Hash similar items into the same bucket
   - Great for cosine / Jaccard distance

2. **Tree-based methods (e.g., KD-trees)**
   - Works well in low dimensions

3. **Graph-based methods (e.g., HNSW)**
   - Build navigable graphs for efficient search
   - State-of-the-art on many tasks

🔍 Often used via libraries like **FAISS**, **Annoy**, **ScaNN**



How ANN Works (Visually)

Using ANN in Practice

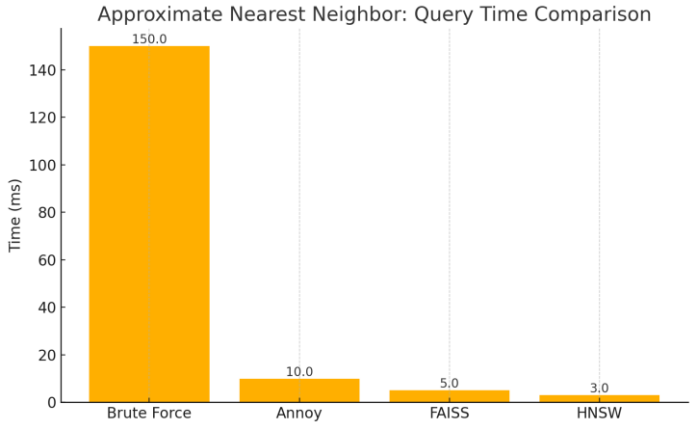
✅ Use ANN when:

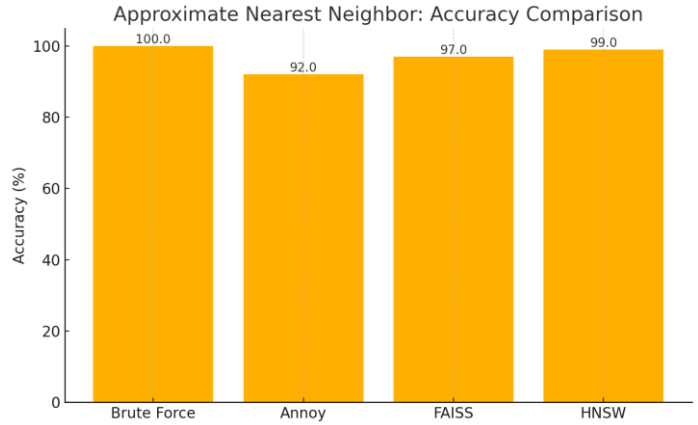
- You have large datasets
- Exact NN is too slow
- Some accuracy tradeoff is acceptable

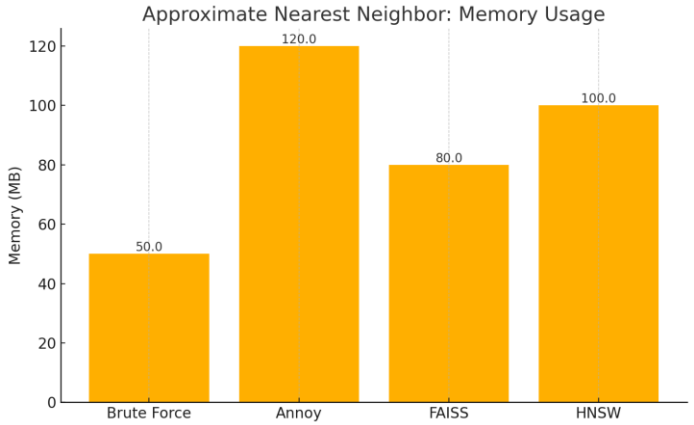
⚠️ Be careful if:

- You need perfect results (e.g., medical diagnoses)
- The dataset is small (brute force may be fine)

💡 **Next year**
Combine ANN with dimensionality reduction (e.g., PCA, UMAP) for best results!



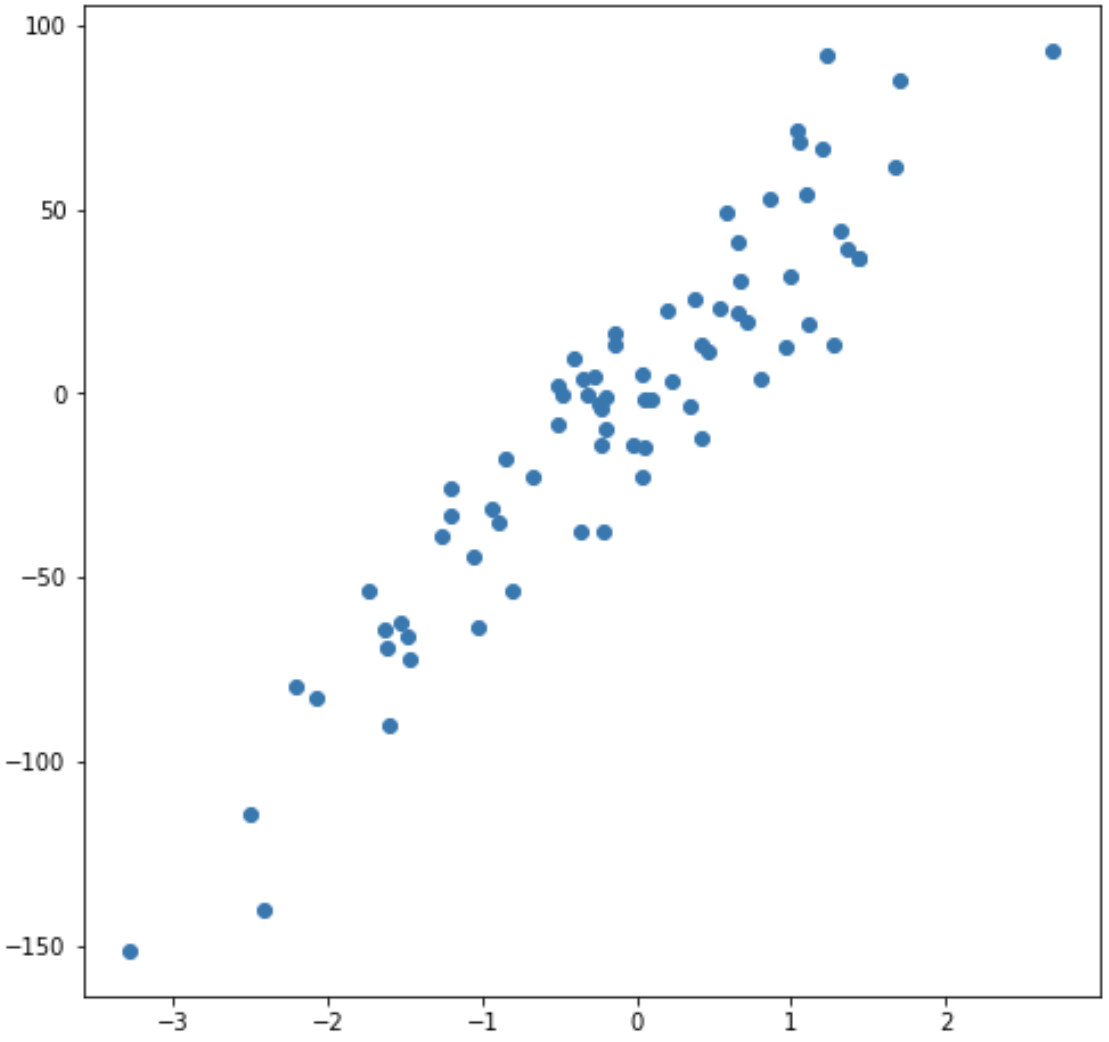




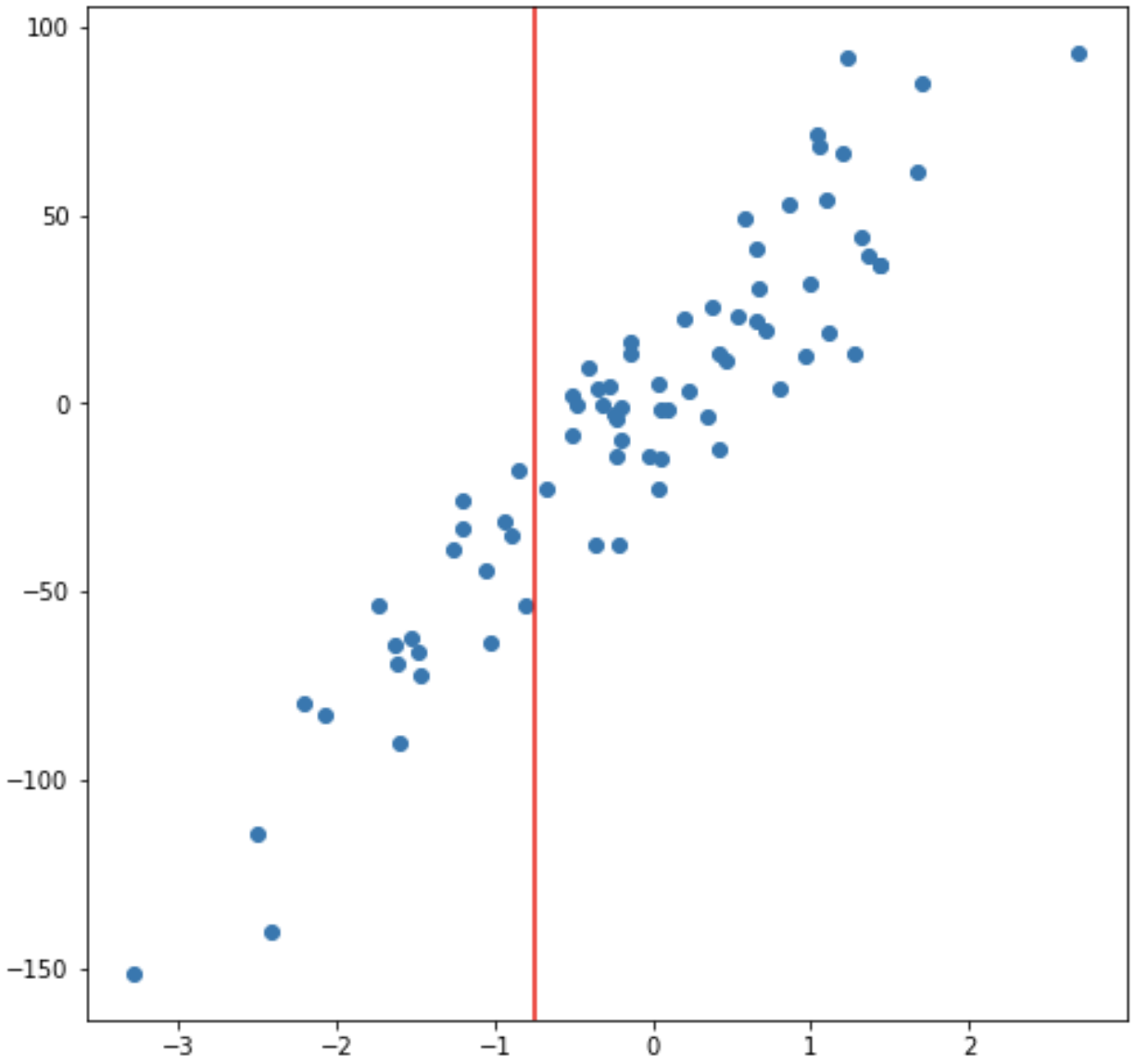
Comparison – 1M points, 128 features, k=1

K-NN for regression

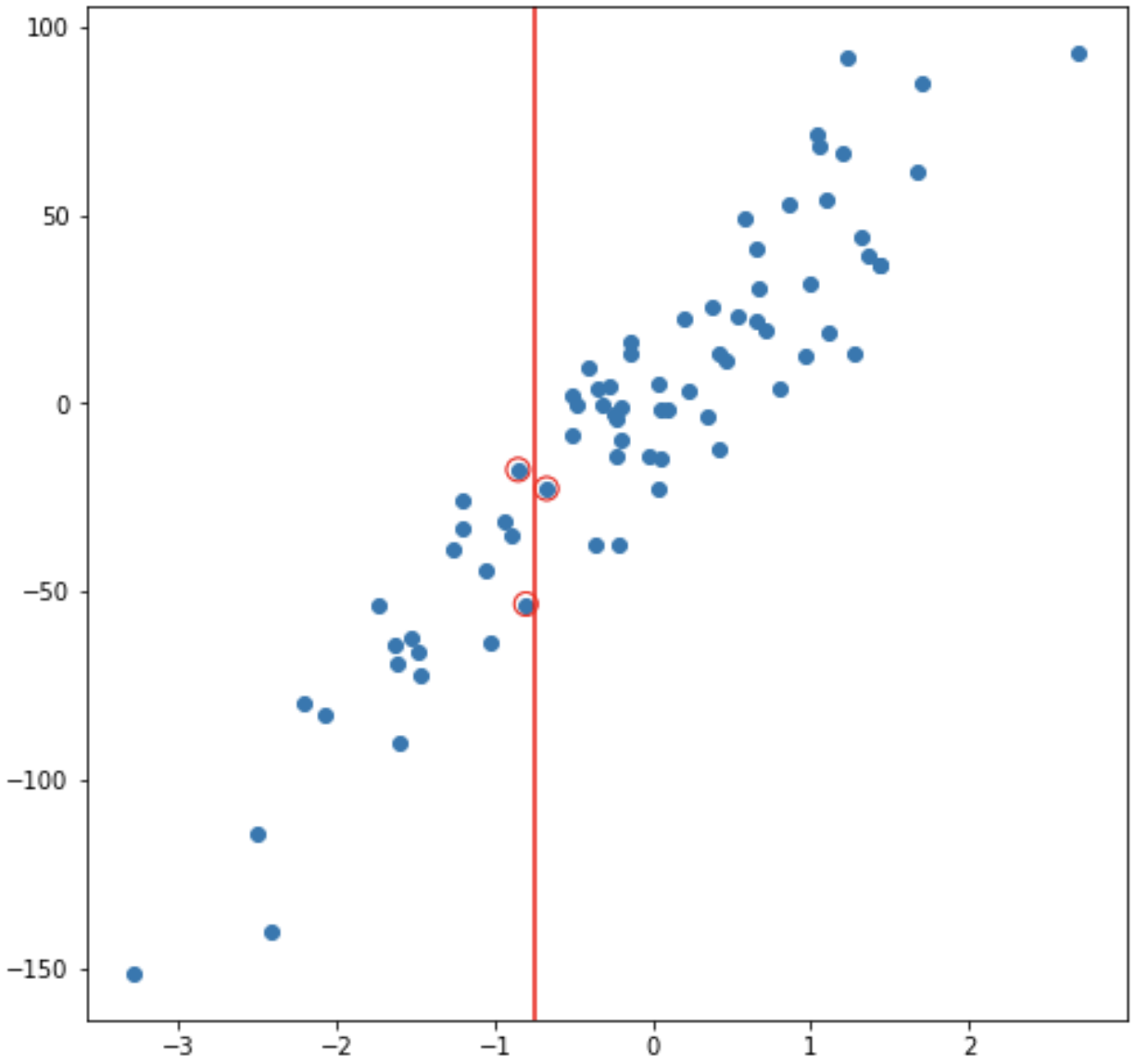
BASED ON TEXT BY MAX MILLER



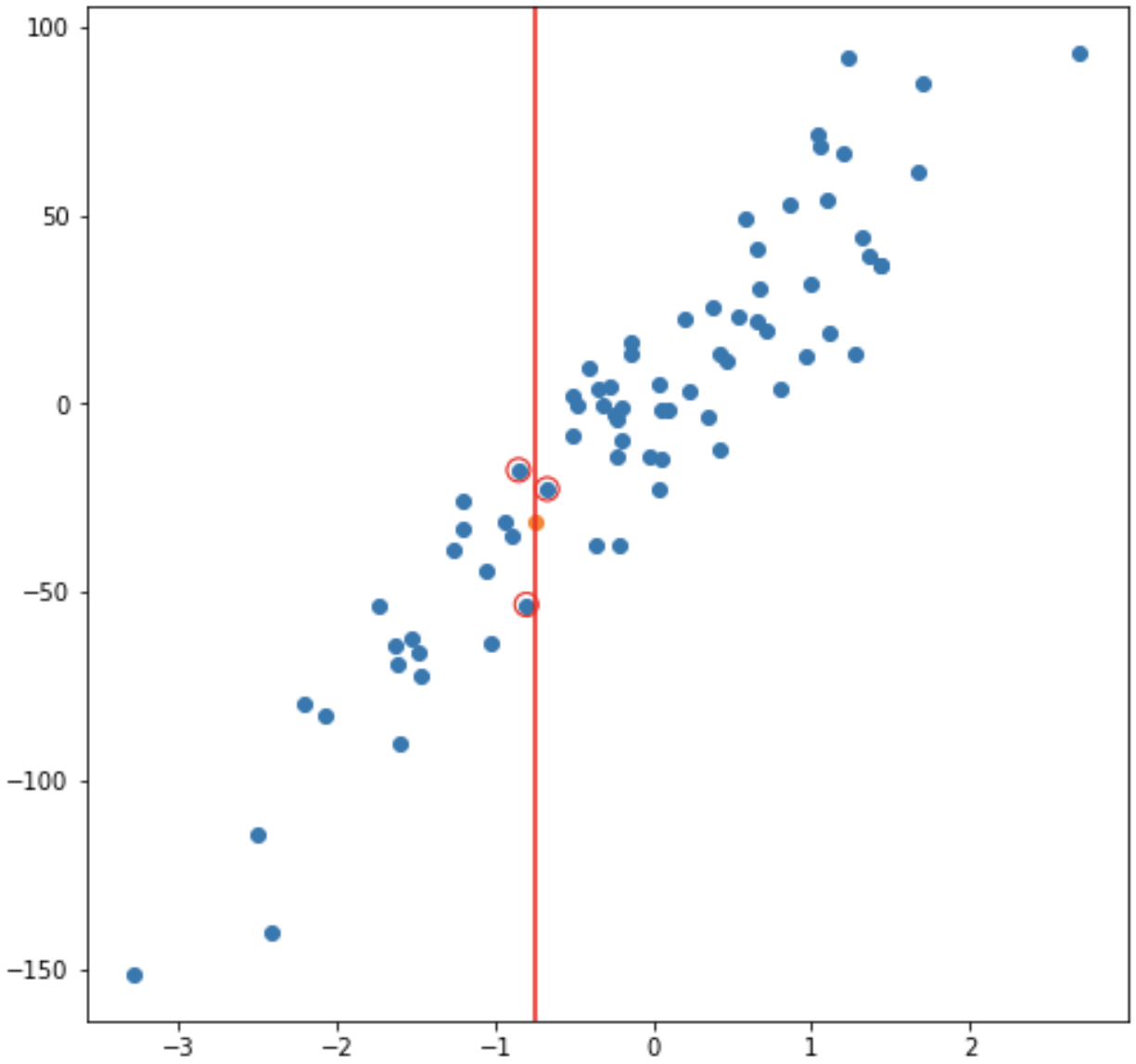
Simple regression example



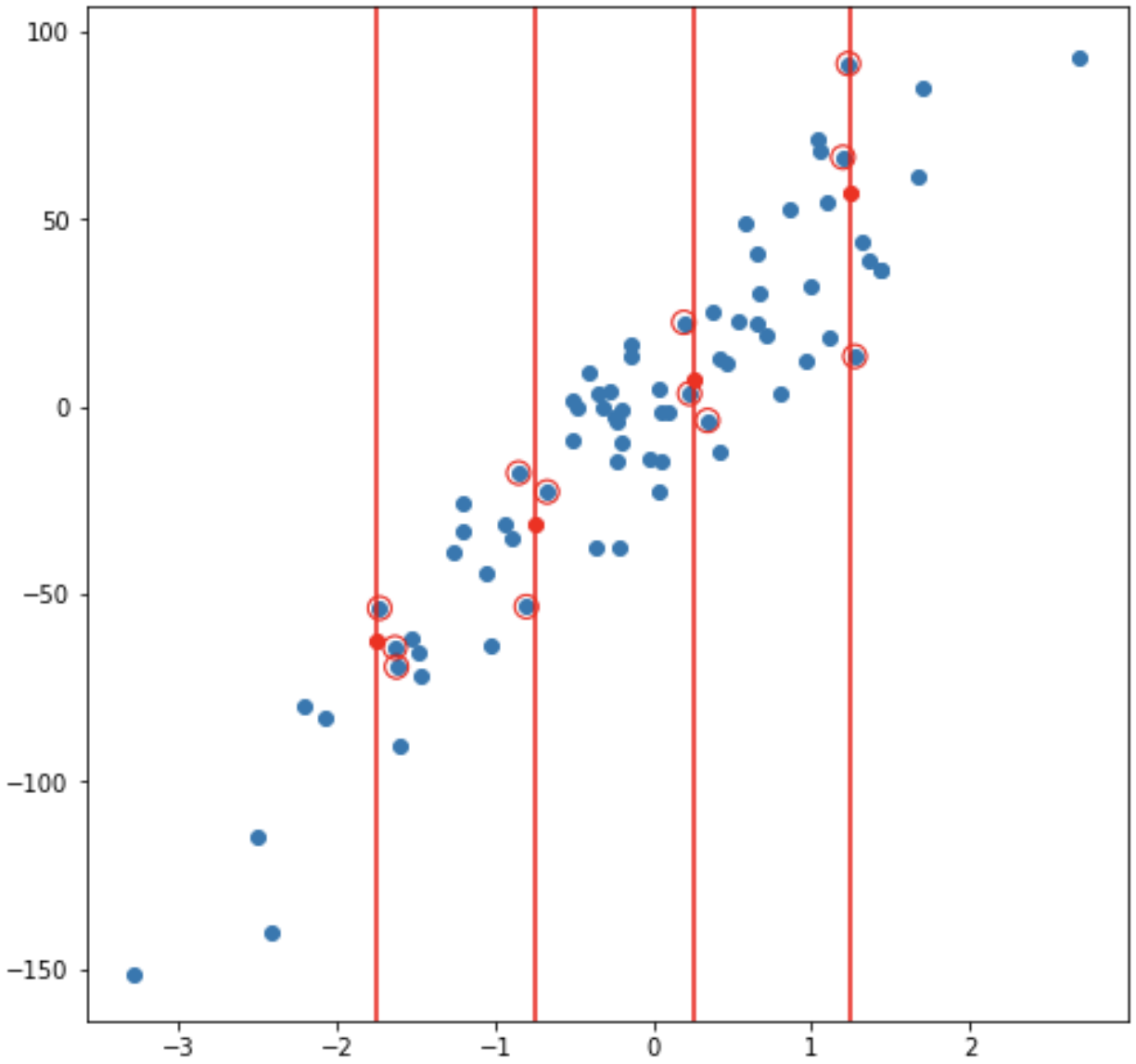
Avoid using OLS



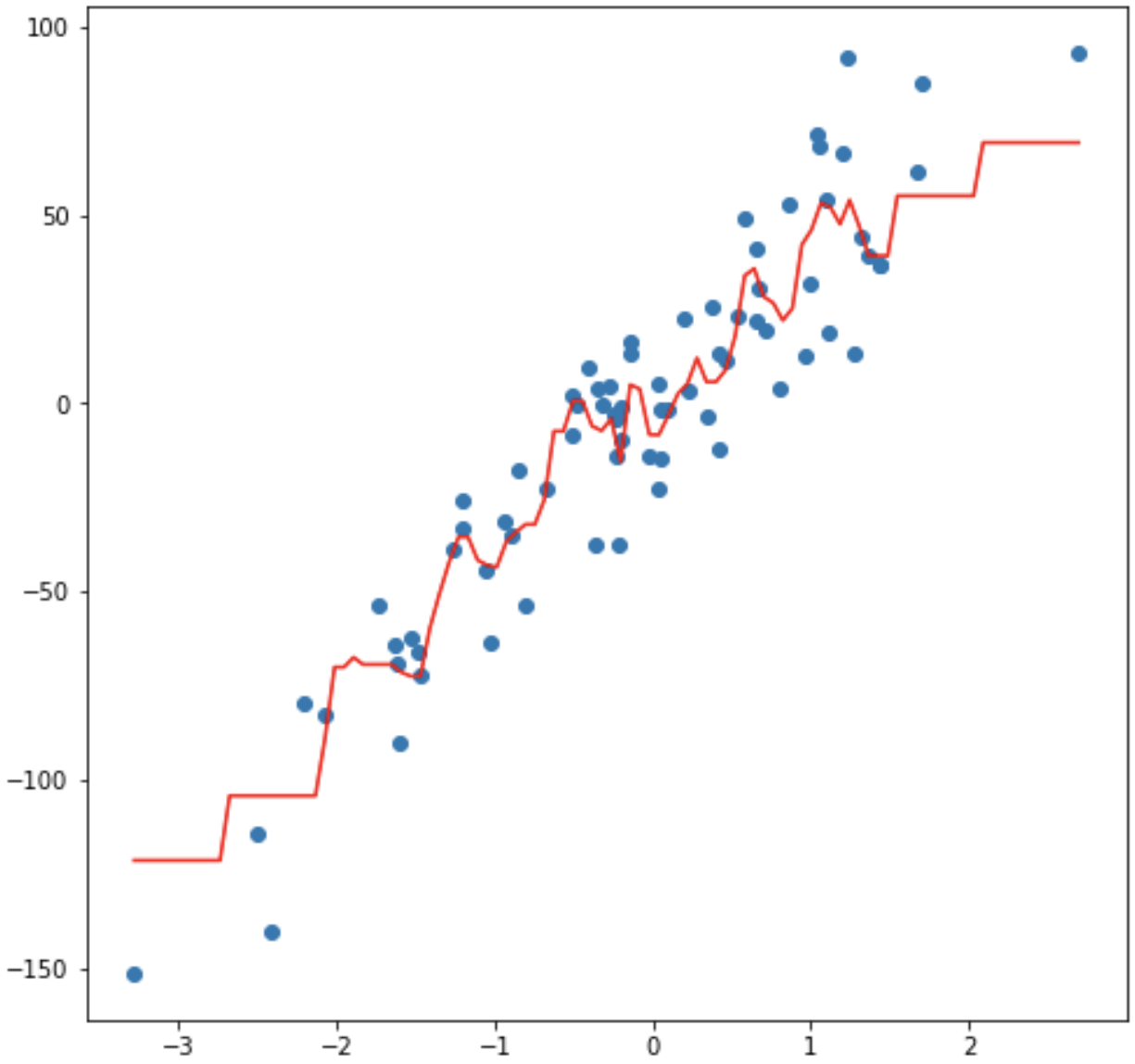
Predict based on the nearest neighbors



Using the average



Predict whatever you want…



We can end up with a regression line